I S P

Interface Segregation Principle

Many small interfaces beat one fat one

DEFINITION

Clients should not be forced to depend on methods they don't use.

Interfaces belong to the client that uses them, not the class that implements them. A "fat" interface bundles methods that different callers need separately, so each caller is coupled to code it never touches. ISP says carve interfaces along usage lines — small role interfaces, one per kind of client — and let a class implement several of them.

WHY IT MATTERS

Depending on a method you never call still costs you: when its signature changes you recompile, re-mock it in tests, and risk breakage for a reason that has nothing to do with you. Worse, fat interfaces force implementers to stub methods they cannot honor with empty bodies or "not supported" throws. Narrow interfaces keep clients insulated and implementations honest.

Vasyl Krupka · Ukraine

How it works

Role interface	A small, client-specific contract (Printer, Scanner). It names exactly what one kind of caller needs — nothing more.
Client	Code that depends on one role. It sees only the methods it uses, so unrelated changes cannot reach it.
Implementer	A class that composes several role interfaces (an all-in-one device is Printer and Scanner). It opts into only the roles it truly supports.

HOW TO APPLY

1. List the distinct clients of a type and what each one actually calls.
2. Split the fat interface into one role interface per client need.
3. Have classes implement only the roles they genuinely support — no stub or throwing methods.
4. Type each consumer against the narrowest role it needs, not the concrete class.

LITMUS TEST

Does any implementer leave methods empty or throwing "not supported"? Does any client use only a slice? Either way the interface wants splitting.

Vasyl Krupka · Ukraine

— Smell → fix

One device interface forces a simple printer to fake what it can't do:

```
interface Machine { print(): void; scan(): void; fax(): void }
class SimplePrinter implements Machine {
print() { /* ok */ }
scan() { throw new Error('not supported') } // forced
fax() { throw new Error('not supported') } // forced
}
```

↓ SPLIT INTO ROLE INTERFACES

```
interface Printer { print(): void }
interface Scanner { scan(): void }
class SimplePrinter implements Printer { print() {} }
class AllInOne implements Printer, Scanner { print(){} scan(){} }
```

Each client now depends only on the role it needs. A multifunction device composes several role interfaces; a simple one implements just Printer.

USE WHEN

- ✓ A "god" interface forces empty or throwing methods
- ✓ Different clients need different slices

AVOID WHEN

- ▲ Shattering into so many interfaces that cohesion is lost

Vasyl Krupka · Ukraine

— Trade-offs & pitfalls

PROS

+ Lower coupling
+ Implementations stay honest

CONS

− More types
− Risk of over-segmentation

COMMON PITFALLS

▲ Over-segmenting into one-method interfaces everywhere — you trade fat interfaces for type sprawl and lost cohesion.
▲ "God" header interfaces that grow as a dumping ground because adding a method is easier than adding a type.
▲ Leaking the concrete class as the parameter type, so clients still see methods they don't need.

WHERE YOU SEE IT

- Node streams: Readable vs Writable — a consumer accepts only the half it uses.
- Typing a function to a structural slice ({ id: string }) instead of the whole entity.
- A repository split into a read port and a write port so query code can't mutate.

SEE ALSO

SRP	the same cohesion idea, applied to interfaces from the client side
DIP	small client-owned interfaces are the abstractions to depend on
LSP	narrow contracts are easier to satisfy without surprises
High Cohesion	GRASP’s framing of keeping a type’s members related

Vasyl Krupka · Ukraine

— Interview & sources

ISP vs SRP — what's the difference?

SRP is about a class having one reason to change; ISP is about clients not depending on more of an interface than they use. Same instinct, different target.

Can you over-apply ISP?

Yes — shattering into one-method interfaces destroys cohesion and floods the codebase with types. Split by client role, not per method.

Where does ISP show up in TypeScript daily?

Accepting the narrowest structural type a function needs (e.g. { id: string } instead of the full entity) is ISP in practice.

Why does ISP cut rebuild and test churn?

A client coupled only to a small role recompiles and re-mocks only when that role changes. Fat interfaces drag unrelated clients into every change.

REMEMBER

No client should be forced to depend on methods it never calls.

SOURCES

Robert C. Martin — "The Interface Segregation Principle" (C++ Report, 1996)

Robert C. Martin — "Agile Software Development, Principles, Patterns, and Practices" (2003)